ENGINEERING PORTFOLIO SHOWCASE

CryptoTrade Journal

A Production-Grade Full-Stack Analytics Platform

JWT & Bcrypt Security | Avg Cost Basis Engine | Role-Aware UI | Deployed & CI/CD Ready

---

Avula Niranjan Reddy • niranjan023a@gmail.com •

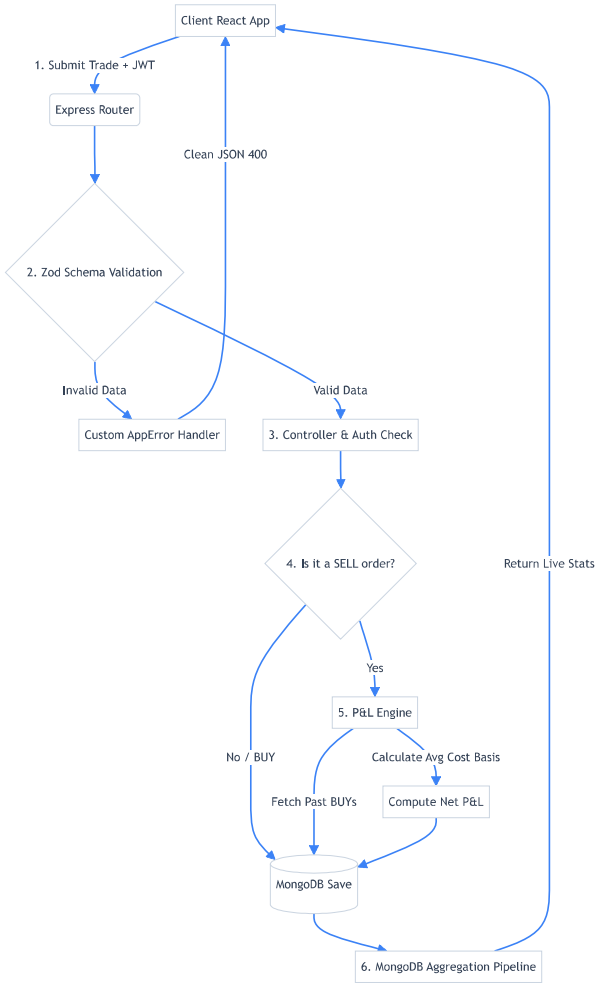
github.com/Niranjan945/TradeMe

The Story & The Problem

> Most retail crypto traders live in **spreadsheet chaos**. A trader might buy Bitcoin on Binance, sell some Ethereum on Coinbase, and accumulate altcoins on decentralized exchanges. By the end of the month, attempting to calculate actual net profit is nearly impossible. Nobody really knows if they made money or just *felt* like they did.
>
> I realized that a simple "notes app" wouldn't solve this. Traders needed an automated system where they simply input their BUY and SELL actions, and an algorithmic engine handles the complex math behind the scenes.
>
> **The Portfolio Objective:** I built CryptoTrade Journal to solve this exact real-world problem. However, my primary goal was to construct a robust, production-deployed web application that acts as a comprehensive showcase of my full-stack engineering skills. I wanted to move beyond basic CRUD tutorials and implement real financial algorithms, strict API boundary validation, and role-based architectural patterns.

Application Architecture & Data Flow

To demonstrate a clear separation of concerns, the application follows a strict data flow. Validation occurs at the exact moment data enters the backend, ensuring the database is never polluted with malformed financial records.



Live Demos & Interactive API Testing

This project is fully deployed. To provide transparency into the backend architecture, I have integrated Swagger for live API testing.

Resource	URL	Purpose for Viewer
Frontend Web App	trade-frontend-l0to.onrender.com	Experience the React SPA and responsive UI.
Interactive API Docs	trademe-1.onrender.com/api-docs	Inspect routes, schemas, and test backend logic directly.
Backend API Root	trademe-1.onrender.com	Base URL for the Node.js service.

Test credentials — Admin login: ***niranjan023a@gmail.com*** */ Password:* ***admin@**1234***

Project Directory Structure

The repository is organized as a monorepo containing both the React frontend and the Express backend, ensuring strict separation of concerns while keeping full-stack development streamlined.

```
        TradeMe / Project Root


TradeMe/
├── client/                 # Frontend Application (React 19 + Vite)
│   ├── src/
│   │   ├── components/     # Reusable UI elements (Modals, Tables, Nav)
│   │   ├── context/        # Global state (AuthContext.jsx)
│   │   ├── pages/          # Route views (Dashboard, Login, Admin)
│   │   ├── services/       # API communication (Axios instances)
│   │   └── App.jsx         # Main routing configuration
│   ├── tailwind.config.js  # Utility styling configuration
│   └── package.json
│
└── server/                 # Backend Application (Node 22 + Express)
    ├── src/
    │   ├── controllers/    # Core business logic (authController, tradeCon
    │   ├── middlewares/    # JWT verification, global error handler
    │   ├── models/         # Mongoose schemas (User, Trade)
    │   ├── routes/         # Express route definitions (v1 API)
    │   ├── utils/          # P&L calculation engine, Custom AppError wrapp
    │   └── server.js       # Application entry point & DB connection
    ├── swagger.yaml        # OpenAPI documentation configuration
    └── package.json
```

Tech Stack & Dependencies

💻 Frontend Stack (Client)

Built for speed and responsive UI.

`react@19` `react-dom` `react-router-dom@7` `vite` `axios` `tailwindcss` `lucide-react` `date-fns`

⚙️ Backend Stack (Server)

Built for robust data validation and security.

`express@4` `mongoose@8` `zod` `bcryptjs` `jsonwebtoken` `cors` `dotenv` `swagger-ui-express`

Local Development & Setup Commands

To run this application locally on your machine, follow these steps. Ensure you have Node.js and a MongoDB instance (local or Atlas) ready.

```bash
# 1. Clone the repository
$ git clone https://github.com/Niranjan945/TradeMe.git
$ cd TradeMe

# 2. Setup the Backend
$ cd server
$ npm install

# Create a .env file in the server directory with:
# PORT=5000
# MONGO_URI=your_mongodb_connection_string
# JWT_SECRET=your_super_secret_key
# NODE_ENV=development

# Start the backend server (starts on port 5000)
$ npm run dev

# 3. Setup the Frontend (in a new terminal tab)
$ cd ../client
$ npm install

# Create a .env file in the client directory with:
# VITE_API_URL=http://localhost:5000/api/v1

# Start the Vite development server
$ npm run dev
```

Note: The backend automatically runs a `seedAdmin()` function on startup. Once the server is running, the admin account will automatically be created in your database.

Skill Showcase Highlights

Architecture & Design

- **Production API Design:** Versioned REST routes (`/api/v1/`) allowing for future mobile app integration without breaking web clients.
- **Strict API Boundary:** Input validation happens at the router level using **Zod**, completely decoupling validation from the database.
- **Error Architecture:** Custom `AppError` class and `asyncHandler` wrappers eliminate redundant try-catch blocks and guarantee clean JSON responses.

Product & Algorithms

- **Real Financial Algorithms:** Implements a true *Average Cost Basis* formula. It dynamically tracks previous buy weights against new sell events to calculate legitimate realized P&L.
- **Advanced DB Queries:** Uses complex `$match` and `$group` MongoDB aggregation pipelines to compute total analytics in a single DB round-trip.
- **Role-Aware Unified UI:** Two completely different user experiences (Trader vs. Administrator) run from one shared codebase depending on the JWT payload.